Cell-Based Architecture for Solving Data Integrity in Microservices

1st Supriya Lal
Independent Researcher
Seattle, USA
ORCID: 0009-0006-6207-6678

***Abstract*—Cell-based architecture is gaining traction as a solution to challenges such as fault tolerance, scalability, latency, and organizational alignment. This paper explores how sharing databases between microservices within a cell can reduce data integrity issues associated with distributed transactions and introduces strategies to manage the operational challenges of this approach.**

***Index Terms*—cell-based architecture, microservices, data integrity**

I. Introduction

Cell-based architecture is increasingly being adopted to address challenges such as fault tolerance, scalability, latency, and organizational alignment. Domain-Driven Cell-Based Architecture [1] builds upon these foundations by applying the principles of Domain-Driven Design (DDD) [2], [3] to cell-based architecture, ensuring that the cell boundaries align with business needs. This paper examines how sharing of databases across microservices [4] within domain cells eliminates distributed transactions [5] while addressing operational challenges through RBAC strategies, monitoring approaches and reorganization guidelines.

II. Related Work

Many companies have adopted a cell-based architecture to improve resilience and availability. Slack transitioned its critical user-facing services to a cell-based setup after experiencing partial outages caused by Amazon Web Services (AWS) Availability Zone (AZ) networking failures [6]. Roblox restructured its infrastructure into cells, allowing it to achieve 99.99 percent uptime [7]. Similarly, DoorDash implemented an AZ-based cell architecture that reduced cross-AZ data transfer costs [8]. AWS has published detailed guidelines for the implementation of cell-based architecture [9].

This paper explores an approach where cell-based architecture is applied to microservices, with all services within a cell sharing a common database to eliminate distributed transactions. While shared databases within bounded contexts are established patterns in microservices literature, this paper's contribution specifically addresses the operational challenges of implementing cell-based architecture by: (1) proposing RBAC strategies for maintaining service boundaries despite database sharing, (2) introducing the concept of monitoring budgets for managing inter-cell dependencies and (3) describing when and how cells should be reorganized based on coupling patterns. These mechanisms provide a framework for making the shared-database approach more manageable at scale, addressing practical challenges that existing DDD practices alone do not fully address.

Alternative approaches to managing data consistency in microservices have been extensively studied. The Saga pattern [10] orchestrates distributed transactions through compensating actions, providing eventual consistency across service boundaries but introducing complexity in managing compensating transactions and handling partial failures. The outbox pattern ensures reliable event publishing for event-driven architectures [11], while event sourcing and CQRS patterns separate read and write models to manage consistency [12]. These event-driven approaches require careful management of eventual consistency. Modular monoliths [13] represent another approach, maintaining logical service boundaries within a shared deployment and database, offering strong consistency and low complexity but sacrificing the independent deployability of microservices.

The cell-based database sharing approach presented in this paper attempts to balance these trade-offs by providing strong consistency within domain boundaries (cells) while maintaining service autonomy through operational strategies.

III. Proposed Solution

Distributed transactions hinder data integrity by requiring complex algorithms, such as the two-phase commit [14], to uphold atomicity and consistency while managing partial failures across multiple services and databases. By eliminating distributed transactions, inserts, deletes, and updates can occur within a single database transaction, ensuring an all-or-none approach that improves data integrity.

This paper suggests an approach in which all services within a single cell share a common database which eliminates or reduces the need for distributed transactions. This is because, in a domain-driven cell-based architecture, tightly coupled domains are confined to a single cell. Therefore, a business workflow is ideally restricted to one cell, which limits communication between services to a single cell. In addition, communication between services in different cells should be minimized by allowing only loosely coupling between cells.

SoutheastCon 2026 | 979-8-3315-4642-7/26/$31.00 ©2026 IEEE | DOI: 10.1109/SOUTHEASTCON63549.2026.11476409

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 08:52:46 UTC from IEEE Xplore. Restrictions apply.
979-8-3315-4642-7/26/$31.00 ©2026 IEEE

This would prevent strong consistency or atomicity in inter-cell transactions.

Let's consider an example as shown in Fig. 1. An inventory management domain consists of three services that manage car listings, track inventory, and handle car suppliers, requiring strong transactional guarantees such as atomically updating inventory and supplier records when a car is purchased. The car servicing management domain handles booking appointments, assigning technicians, and managing car servicing reviews, each through a dedicated service, also requiring atomic updates (e.g., booking a service and updating technician assignments together). When infrequent cross-domain interactions occur, such as scheduling a service appointment while purchasing a car, strict atomicity across domains is not required.

Inventory management and car servicing management are split into separate cells, with each cell sharing a common database internally. Loose coupling between cells minimizes the need for distributed transactions, which require transactional guarantees across cells, while still ensuring atomicity and consistency within each domain.

In a traditional microservices architecture, it is usually recommended that each service has its own database [12] for the following reasons:

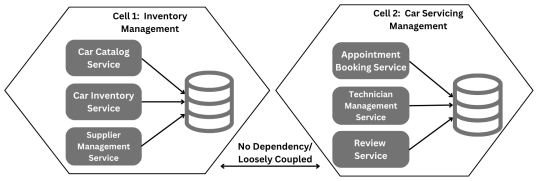


Fig. 1. Cell Based Architecture with Common Database

- `Autonomous development`: Services with separate databases are less coupled, allowing teams to iterate on their services without coordinating with other teams or risking breaking changes in a shared database.
- `Fault isolation`: Separate databases improve fault isolation by containing failures such as data corruption to the specific service's database without affecting others. This ensures that issues in one service do not cascade across the system.
- `Security and access control`: Separate databases enable security and access control by limiting each service's access to its own database. This reduces the risk of unauthorized data exposure.
- `Clear data ownership`: When each service is owned by a single team, separate databases enable teams to make independent decisions about schema design, scaling, and optimization without relying on other teams.

While sharing a database can improve transactional guarantees, it increases coupling between services and undermines microservices benefits like autonomy, data ownership, security, and fault isolation. To mitigate these risks in a cell-based architecture with a shared database, consider the following strategies, as illustrated in Fig. 2:

A. Step 1 - Determine Cell Boundaries

DDD is essential for ensuring that a highly coupled system is encapsulated within a single cell. To define effective domain boundaries, teams should collaborate closely with business stakeholders and develop a thorough understanding of the business requirements [2]. Once domain boundaries are established, assign each domain to a separate cell.

Each cell represents a complete bounded context, with services acting as cohesive components within that domain. The cell owns the database, which prevents unrelated services from sharing data without clear boundaries.

When business domains overlap, tightly coupled domains requiring strong consistency and atomicity should be placed within the same cell, allowing operations to be handled within a single database transaction. In contrast, loosely coupled domains that do not require transactional guarantees can be assigned to different cells. This prevents cells from becoming too large while still maintaining atomicity and consistency.

Defining clear cell boundaries using DDD would set the foundation such that:

- `Minimal inter-cell communication`: Communication between cells is limited, avoiding transactions that require strong consistency or atomicity.
- `Autonomous development`: Well-defined domains usually correspond to dedicated teams or closely aligned groups. This enables teams to operate without relying on unrelated teams.
- `Fault isolation`: By restricting each business workflow to a single cell or a small set of related cells, failures are contained and do not cascade into unrelated parts of the system.

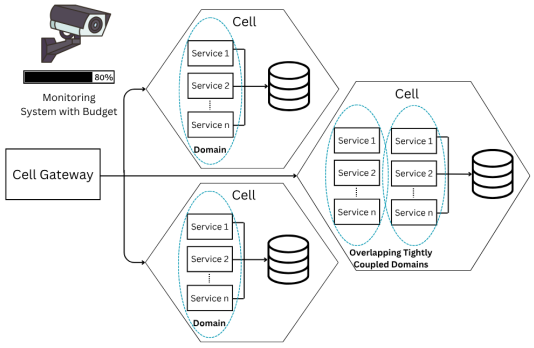


Fig. 2. Cell Based Architecture Implementation Strategy

B. Step 2 - Database Drift Prevention and Autonomy

Within each cell, implement a shared database for all services. To minimize security risk and preserve service boundaries, implement Role-Based Access Control (RBAC), granting each service only the minimum required permissions. Write access to another service's data should be granted directly only when strong transactional consistency is required, and then only for the specific tables or columns necessary. In all other cases, write requests should be routed through

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 08:52:46 UTC from IEEE Xplore. Restrictions apply.

the owning service's API. Read access to data owned by another service should always be performed via the owning service's API, never through direct database queries. Cross-cell communication should always occur through owning service APIs. This approach restricts direct database access to only those scenarios where data consistency is essential. By routing most read and write operations through service APIs, it ensures that each service retains control over its own data, thereby maintaining clear ownership and encapsulation. It also enforces service boundaries, as services cannot bypass business logic by accessing each other's tables directly. Additionally, minimizing direct database access helps mitigate security risks by reducing the potential for unauthorized data access.

Beyond RBAC, maintaining service boundaries requires schema governance. Each service should own distinct tables following naming conventions (e.g., servicename-tablename), with migration ownership restricted to the owning team. Automated checks should validate that services only access their owned tables directly, with any cross-service access requiring explicit approval.

C. Step 3 - Monitor Cross-Cell Interactions

Cross-cell interactions, especially those requiring transactional guarantees, can undermine the benefits of shared databases and independent deployability in cell-based architecture. Teams should implement monitoring to track two key metrics: the percentage of API calls crossing cell boundaries and the percentage of write operations requiring cross-cell coordination. While cross-cell read operations may be acceptable at higher rates, cross-cell write operations should remain minimal as these require distributed transaction handling. When cross-cell writes exceed this threshold, it indicates that tightly coupled domains have been incorrectly split across cells.

D. Step 4 - Reorganize Cells

Cells may need to be reorganized when the budget for inter-cell communication is exceeded or when a single cell becomes very large. This reorganization may involve merging multiple cells or splitting a single cell, and requires the following steps. 1) Redefine which services belong in each cell based on updated domain boundaries. 2) Update any direct database writes between services to instead route through the service APIs, and revoke any direct database write permissions when splitting services across cells. 3) Migrate database tables to the new cell using data migration tools like mysqldump [15], ensuring data accuracy. 4) Deploy services to the new cell, configuring RBAC to manage shared database access. 5) Conduct end-to-end testing to verify that all functionality remains correct.

IV. Challenges

Firstly, defining correct domain boundaries can be challenging, and inaccurate boundaries may undermine cell isolation. To mitigate this, invest significant effort in domain modeling before implementation, using techniques such as event storming to effectively establish clear boundaries. Secondly, sharing a database among services can create dependencies that may be hard to predict, which can make it difficult to change one service without impacting others. To address this, enforce service boundaries by limiting direct database access to only those cases that require transactional guarantees. Thirdly, as cells grow larger or new ones are added, they may become unmanageable, requiring reorganization. Additionally, software companies often evolve rapidly, requiring adjustments to cell boundaries. However, changing cells, specifically with a shared database can be resource-intensive. To minimize risks, maintain strict service boundaries using RBAC and plan data migrations carefully to prevent data inaccuracies. Fourthly, establishing a correct budget for intercell communication is not trivial and can lead to inefficient decisions regarding cell reorganization. If the budget is set too low, it might trigger unnecessary reorganizations. Setting the budget too high would allow excessive intercell dependencies. To manage this, base the initial budget on observed call patterns, and regularly review and adjust it to align with evolving architecture. Fifthly, shared databases create cell-level single points of failure and potential throughput bottlenecks. Mitigation strategies include database replication for high availability [16], table partitioning to isolate service workloads [17], and resource quotas to prevent noisy-neighbor problems [18].

Each organization should carefully assess whether the benefits of adopting a domain-driven cell-based architecture outweigh the trade-offs for their particular use case. Sharing a database among services within a cell is most advantageous when: 1) Services are tightly coupled and require strong transactional guarantees. 2) The domains can be distributed across cells such that there is minimal communication between them. 3) The domain is well-understood and stable, reducing the risk of frequent reorganizations. 4) The cell contains a manageable number of services, and the team structure closely aligns with the cell.

V. Conclusion And Future Work

Sharing databases within cells in a cell-based architecture offers a promising approach to addressing data integrity challenges commonly faced in microservice architectures. However, sharing a database across services has limitations, such as lack of fault isolation, liberal access to data, interdependent development, and unclear ownership. Implementing strategies such as identifying domains using DDD, monitoring cross-cell interactions, using RBAC to manage access, and planning reorganizations can help avoid the pitfalls associated with database sharing.

This paper presents a conceptual framework that establishes a foundation for practitioners to implement shared databases within cell-based architectures. Future research opportunities include analysis comparing transaction latency and failure rates between traditional distributed transactions and the proposed approach, as well as examining the impact on deployment independence and schema coordination overhead.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 08:52:46 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] Y. N. Roa, S. Chaplin, E. Pisani, G. Coleman, and C. Price, "Cell-based architectures: How to build scalable and resilient systems," *InfoQ*, 2024.

[2] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2004.

[3] Uber Engineering, "Introducing domain-oriented microservice architecture," https://www.uber.com/en-US/blog/microservice-architecture/, 2020, uber Engineering Blog, July 23, 2020. Retrieved May 30, 2025.

[4] N. Dragoni *et al.*, "Microservices: Yesterday, today, and tomorrow," *Present and Ulterior Software Engineering*, pp. 195–216, 2017.

[5] P. Helland, "Life beyond distributed transactions: An apostate's opinion," *CIDR*, pp. 132–141, 2007.

[6] C. Bethea, "Slack's migration to a cellular architecture," https://slack.engineering/slacks-migration-to-a-cellular-architecture/, 2023, slack Engineering Blog, August 22, 2023. Retrieved May 30, 2025.

[7] D. Sturman, M. Ross, and M. Wolf, "How we're making roblox's infrastructure more efficient and resilient," https://corp.roblox.com/newsroom/2023/12/making-robloxs-infrastructure-efficient-resilient, 2023, roblox Engineering Blog, December 7, 2023. Retrieved May 30, 2025.

[8] H. Wong and L. Stepanian, "Staying in the zone: How doordash used a service mesh to manage data transfer, reducing hops and cloud spend," https://doordash.engineering/2024/01/16/staying-in-the-zone/, 2024, doorDash Engineering Blog, January 16, 2024. Retrieved May 30, 2025.

[9] Amazon Web Services, Inc., "Reducing the scope of impact with cell-based architecture," https://docs.aws.amazon.com/wellarchitected/latest/reducing-scope-of-impact-with-cell-based-architecture/reducing-scope-of-impact-with-cell-based-architecture.html, 2023, aWS Well-Architected Framework, September 20, 2023. Retrieved May 30, 2025.

[10] H. Garcia-Molina and K. Salem, "Sagas," *ACM SIGMOD Record*, vol. 16, no. 3, pp. 249–259, 1987.

[11] M. Kleppmann, *Designing Data-Intensive Applications*. O'Reilly Media, 2017.

[12] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. O'Reilly Media, 2021.

[13] R. Su and X. Li, "Modular monolith: Is this the trend in software architecture?" in *Proceedings of the 1st International Workshop on New Trends in Software Architecture*, ser. SATrends '24. New York, NY, USA: ACM, 2024, pp. 10–13.

[14] P. Yadav, "Design and evaluation of event-driven architectures for transaction management in microservices," vol. 6, pp. 145–155, July 2020.

[15] *mysql, mysqladmin, and mysqldump*. Berkeley, CA: Apress, 2005, pp. 61–70.

[16] J. Gray and A. Reuter, *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann, 1993.

[17] D. DeWitt and J. Gray, "Parallel database systems: The future of high performance database systems," *Communications of the ACM*, vol. 35, no. 6, pp. 85–98, 1992.

[18] A. Verma *et al.*, "Large-scale cluster management at google with borg," in *Proceedings of the Tenth European Conference on Computer Systems*, 2015, pp. 1–17.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 08:52:46 UTC from IEEE Xplore. Restrictions apply.